Harwise: A Local AI Agent for Paddy and Vegetable Farmers in Sri Lanka

LLM Engineering Assignment: Building an AI Agent with Ollama

Group members: D.D.A.Nipun ,J.P.D.Madhusankha
Repository: https://github.com/damithmadhusankha-online/harwise-ai
Date: October 2026

1. Problem and Scenario

Farmers in Polonnaruwa grow paddy in the Maha and Yala seasons and often grow vegetables such as brinjal, long beans, chillies and pumpkin on upland plots. Near harvest they face a chain of linked decisions with little information. Should the crop be sold now or kept? Will rain spoil drying or transport? Is the buyer's moisture deduction fair? Which crop should be sold first when cash is needed this week? What should be planted next season with the water available? Traders and millers usually hold more market information than the farmer, so the farmer often accepts the first price offered.

Harwise is an AI agent that works through the same steps a careful adviser would: check prices, check the weather, compare selling options, and decide what to sell first. It is not a chatbot that only talks. A question like the one below cannot be answered with one lookup, because each step depends on the result of the previous tool call:

> *"I harvest 3 acres of paddy in 12 days, moisture about 18%. I also have 400 kg brinjal. I need Rs. 60,000 this week. What do I sell first, and should I store the paddy?"*

Why a local model. Farm finances are private, many farmers have weak internet, and the assignment requires that the core agent does not use paid cloud APIs. Harwise runs its LLM through Ollama on the user's own computer and keeps all farm data there. The only network call is the free Open-Meteo weather forecast, which has a cached copy and a sample-data fallback.

Scope. One district (Polonnaruwa), five crops (paddy, brinjal, long beans, chillies, pumpkin), and decision support only. The agent shows expected, best-case and worst-case results and never promises an outcome. The farmer makes the decision.

2. Architecture and Model Choice

Figure 1 shows the system. The farmer types a question in the command line. The agent loop sends the conversation and the tool descriptions to the local model in Ollama. The model replies with either a final answer or one or more tool calls. The loop gives each tool call to the tool registry, which validates the arguments, runs the matching Python function, logs the call and result, and returns the result to the model. This repeats until the model answers or the step limit is reached.

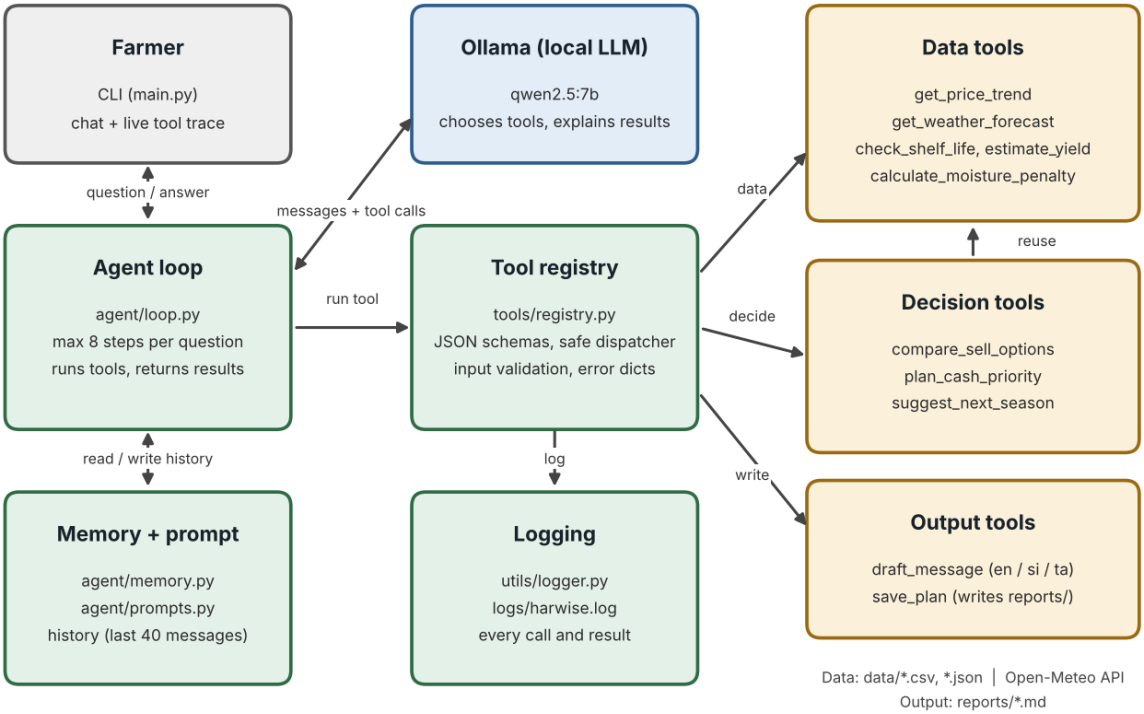


Figure 1. Harwise architecture: agent loop, local model, tool registry, tool groups and data flow.

Model. We use **qwen2.5:7b** (7 billion parameters) through Ollama. We chose it because it supports tool calling, follows instructions well for its size, and runs on an ordinary laptop. Smaller models such as qwen3:4b and llama3.2:3b are faster but usually make more tool-calling mistakes.

Design principle: the LLM decides, the code calculates. Small local models make arithmetic mistakes, so every price, cost and total is computed by tested Python functions. The model only chooses the next tool and explains the results in plain words. Section 5 shows the real failure that led to this rule.

3. Tool Designs

Harwise has ten tools. Each is a plain Python function with a docstring and typed parameters, plus a JSON schema in tools/registry.py that tells the model when to use it. A test checks that every tool has a docstring and type hints.

Tool	Purpose	Key parameters
get_price_trend	Latest, lowest, highest and average price per kg, and direction (rising, falling, flat).	crop, days
get_weather_forecast	Rain forecast and a rain_risk level (low, medium, high). Open-Meteo, then cache, then sample.	location, days
check_shelf_life	How many days a crop can wait and how urgent selling is.	crop
estimate_yield	Expected, low and high harvest in kg from acres.	crop, acres, own_yield_kg_per_acre
calculate_moisture_penalty	Paddy price cut and drying cost for wet paddy.	moisture_pct, kg, rain_risk
compare_sell_options	Expected, best-case and worst-case net income for selling now, selling at Dambulla, or drying and storing.	crop, kg, moisture_pct, wait_days, rain_risk, own_cost_per_kg, pmb_price_per_kg
plan_cash_priority	Decides what to sell first, and how many kg, to reach a cash target.	items (crop, kg), cash_needed
suggest_next_season	Ranks crops for the next season by water need and price direction.	water_level, season, acres
draft_message	Drafts (never sends) a buyer or driver message in English, Sinhala or Tamil.	recipient, purpose, crop, kg, language, when

Tool	Purpose	Key parameters
save_plan	Saves the final plan as a markdown file in reports/.	title, content

How the tools meet the requirements

- **Persistent data:** tools read data/prices.csv, crops.json and assumptions.json. save_plan writes a markdown plan to reports/.
- **External API:** get_weather_forecast calls the free Open-Meteo API and falls back to a cached copy, then to sample data, so the demo works offline.
- **Validation and errors:** every tool checks its inputs (crop names with typo suggestions and aliases such as "rice", quantity ranges, moisture from 0 to 100, allowed choices). A bad input returns ok=false with a clear message instead of raising an exception. The registry also catches wrong argument names and unexpected crashes.
- **Realistic mock data:** the sample prices are labelled as sample data and show different trends (paddy rising, brinjal and pumpkin falling).
- **Safety:** draft_message only writes text. Harwise never sends messages, books anything or makes payments.

Real sample outputs (captured by running the tools)

get_price_trend(crop='paddy', days=14)

```
{"ok":true,"crop":"paddy","period":"2026-10-10 to
    2026-10-22","latest_price_per_kg":129,"lowest_price_per_kg":123,"highest_price_per_kg":129,"av
    erage_price_per_kg":125.6,"change_percent":4.9,"direction":"rising"}
```

compare_sell_options(crop='paddy', kg=5400, moisture_pct=18, wait_days=14, rain_risk='high')

```
sell_now_to_trader: expected Rs. 628,612 (worst case Rs. 599,374, best case Rs. 628,612)
dry_and_store_14_days: expected Rs. 641,022 (worst case Rs. 610,379, best case Rs. 641,022)
warning: No PMB price given, so PMB option was skipped. Ask the farmer for the current PMB price.
```

plan_cash_priority(items=[paddy 5400 kg, brinjal 400 kg], cash_needed=60000)

```
brinjal: sell all 400 kg now, about Rs. 52,200 after buyer discount (perishable, do not hold)
paddy: sell about 70 kg now (about Rs. 8,669 after buyer discount) and keep 5330 kg to dry/store
    (needed to reach the cash target)
cash_target_met: true, expected_cash_after_discount: 60,869
```

get_price_trend(crop='brinjall') # invalid input

```
{"ok":false,"error":"Unknown crop 'brinjall'. Known crops: paddy, brinjal, long beans, chillies,
    pumpkin. Did you mean: brinjal?"}
```

4. Agent Loop and Prompt Design

Loop (agent/loop.py). For each user message the loop (1) checks it is not empty and is under 2,000 characters, (2) adds it to the history, (3) calls ollama.chat with the history and the tool schemas, (4) if the reply contains tool calls, runs each one through the registry, adds the results as tool messages and returns to step 3, and (5) returns the text when there are no more tool calls. A limit of 8 steps prevents infinite loops. Ollama errors (server not running, model not pulled) are caught and turned into helpful messages.

Memory (agent/memory.py). The conversation is kept across turns, so follow-ups such as "what if I sell the brinjal at Dambulla instead?" work. History is capped at the last 40 messages so the small model's context does not overflow, and trimming never leaves an orphaned tool result at the start.

System prompt (agent/prompts.py). It defines the role and includes four groups of rules, each added after a real problem in testing: a typical workflow (price, then weather, then compare options, then cash priority); NUMBER RULES (quote tool numbers exactly, never calculate); CASH RULES (only say a cash need is covered if the tool says so, and give exact kilograms to sell now and keep); and LANGUAGE RULES (write in simple English and copy Sinhala lines from the tools). Tools also return a farmer_summary list of ready-to-quote sentences, which reduces the chance that the model recalculates.

Error handling. When a tool returns ok=false, the error text goes back to the model, which can fix the input and retry. Unknown tool names, malformed JSON arguments and tool crashes are all converted into error results, so the loop does not crash.

Logging and trace. Every user message, tool call, argument set and result is written to logs/harwise.log. The terminal also shows a live trace of each step (CALL and RESULT lines), which is the trace shown in the demo video.

5. Testing

Automated tests. 28 unit tests (python tests/run_tests.py, or pytest) cover normal results, validation failures and edge cases for every tool. One example is a long wait that is not realistic for long beans. They also cover the dispatcher (unknown tools, bad arguments, JSON-string arguments). The network is disabled in tests with HARWISE_OFFLINE=1. Unit tests cannot check the language model, so we also ran the full agent by hand with Ollama using the example prompts.

What failed and how we fixed it

Problem found in testing	Cause	Fix and result
The agent wrote "Rs. 754,600" for 5,400 kg at Rs. 129 per kg. The correct figure is Rs. 696,600, and it also contradicted the tool's net figure of Rs. 628,612.	The small model did its own arithmetic.	Tools now return farmer_summary sentences, and NUMBER RULES were added to the prompt. In the re-run the figures matched the tools (Rs. 696,600 and Rs. 58,000).
The agent said brinjal covers the Rs. 60,000 need and advised storing all the paddy, but brinjal is worth only Rs. 58,000 before costs.	plan_cash_priority could only say sell or wait, and used gross values.	The tool now plans a partial sale after the buyer discount: brinjal 400 kg about Rs. 52,200 and paddy 70 kg about Rs. 8,669, total Rs. 60,869. It returns cash_target_met and shortfall. CASH RULES and two tests were added.
When the agent wrote Sinhala itself, the output mixed English letters and meaningless words.	A 7B model writes poor Sinhala, and the first prompt told it to use the farmer's language.	Sinhala summary lines are now generated by code templates (tools/i18n.py) and the prompt forbids the model from writing Sinhala. A test checks the lines contain Sinhala and no English letters.
test_weather_offline_sample failed on a team member's computer.	The test assumed no cache file, but a live run had saved a cached forecast, which the tool correctly prefers.	The test now accepts cached or sample data, and the cache file was added to .gitignore.

6. Limitations and Future Improvements

Limitations

- The data is sample data and the cost assumptions are placeholders, not official figures. Results show the method, not real market outcomes.
- Best and worst cases use the lowest and highest price in the last 14 days. This is not a price forecast.
- The Paddy Marketing Board price is not built in. The farmer must supply it.
- The small model can still skip a tool, ignore a warning or phrase advice badly. Prompts reduce this but do not remove it.
- Sinhala and Tamil output comes only from templates, and the model understands Sinhala input poorly, so questions work best in English.
- The agent has not yet been validated with real farmers, and the interface is the command line only.

Future improvements

- Use real HARTI or Dambulla price data and cost figures confirmed in interviews with farmers.
- Add a Streamlit or voice interface for farmers who do not type much.
- Add an automatic number checker that rejects any answer containing a figure not returned by a tool.
- Compare the agent's suggestions with the prices that actually followed, and try a larger model.

- Extend to more districts and crops.

Appendix A. Use of AI Coding Assistants

As the assignment permits, the team used Claude, an AI assistant from Anthropic, to brainstorm and choose the scenario; to draft the project structure, tool functions, agent loop and unit tests; to diagnose the problems in Section 5 after we ran the agent and reported its output; and to draft this report from the project files. All members have read the code and can explain it, including the agent loop line by line.

Credits. Ollama and the ollama Python library; the Qwen2.5 model; the Open-Meteo free weather API. Sample prices and assumptions in the repository are placeholders written for this project.